

Как компьютер генерирует текст?

Пилотный параграф – Османов Осман

9 апреля 2026 г.

1 Введение

Ещё начиная со средних веков человечество задумывалось над тем, как сделать ручную работу более эффективной. В XVI веке активно распространилась концепция *разделения труда* между рабочими, а в XX веке эта концепция эволюционировала в концепцию **конвейера** и машинного труда.

Человечество усвоило, что сложные и рутинные дела можно перекладывать на машину. И дело тут не ограничивается заводами. В середине XX века появился первый компьютер ENIAC размером с целое помещение, а в 70-х появились привычные и знакомые нам *персональные компьютеры*. Теперь каждый мог использовать компьютер для составления и печати документов, а также поручать компьютеру нудную вычислительную работу. Активно развивались офисные программы, которые и по сей день упрощают жизни миллионов людей по всему миру.

Однако есть задачи, которые для человека относительно просты, но запрограммировать которые достаточно сложно. Например, перевод с одного языка на другой. Как научить компьютер выполнять такую работу?

В этом параграфе мы сделаем акцент на очень актуальную сегодня задачу – работу с текстом. Попробуем разобраться с тем, как научить компьютер выполнять работу с текстом и понимать его содержимое. Обсудим, как человечество приближалось к тем технологиям, которые существуют в настоящее время, и которые получили широкое распространение, – *большим языковым моделям (БЯМ)*.

2 Как всё начиналось?

Направление в машинном обучении, которое занимается работой с текстом, называется *обработкой естественного языка* (*natural language processing* – *NLP*). Одна из первых задач, которую исследователи хотели решить, – это задача *машинного перевода*.

Самые первые реализации алгоритмов машинного перевода работали преимущественно по принципу *кальки* – когда все слова переводятся независимо друг от друга. Однако такой метод часто неспособен передать смысл сказанного. Например, немецкая фраза *Das ist mir Wurst* («мне всё равно») переводится калькой как «это есть мне колбаса» (см. рис. 1).

DAS	IST	MIR	WURST
ЭТО	ЕСТЬ	МНЕ	КОЛБАСА

Рис. 1: Пример перевода калькой.

Впоследствии алгоритмы становились совершеннее. В 1954 году был проведён совместный эксперимент между IBM и Джорджтаунским университетом, в рамках которого исследователи смогли перевести 60 предложений с русского языка на английский. В память компьютера загружали русские слова с переводами их форм, а также присваивали этим словам коды (см. рис. 2).

Обратите внимание на то, что приведённый словарь — это не просто список слов с переводом. В этом списке слова подразбиваются на смысловые части: начало слова и его конец. Некоторые слова меняют свой смысл в зависимости от окончания (см. рис. 3).

Лингвист Пол Гарвин предложил алгоритм с шестью правилами перевода с русского языка на английский. Эти правила реализовывались с помощью кодов, которые присваивались каждому слову. Например, код 21 у слова означает, что при переводе нужно поменять его местами со следующим словом. Это связано с грамматическими особенностями английского языка:

Приготавливается крахмал (21) → Starch (21) is prepared.

Описанный эксперимент оказал большое влияние на развитие языковых моделей. В частности, идея о хранении словаря и дроблении

<u>ENTRY</u>	<u>EQUIVALENTS</u>	<u>CODES</u>	
OTDYELYENIYE	DIVISION	121	242
	SQUAD	122	242
OTNOSHYENI-	RELATION	151	
	THE RELATION	152	
-OV	OF	131	222
	--	132	222
-OYE	--		
POGOD-	WEATHER		
POLUCHAYET	GETS		
POLYITYICHYESK-	POLITICAL		
PONYIMANYIYE	UNDERSTANDING	242	
POSLYEDN-	LAST		
	LATEST		

Рис. 2: Вырезка из словаря.

GRAZHDANSK-	CIVIL		
-I	OF	131	25
	--	132	25
-IM	BY	131	23
	--	132	23
-IMYI	BY	131	222 23
	--	132	222 23
-IX	OF	131	222 23

Рис. 3: Вырезка из словаря. Учёт окончаний.

слов на смысловые части нашла широкое применение и в современных моделях.

3 Современные языковые модели

Сегодня современные языковые модели позволяют решать множество задач. Но, как и человек, языковая модель этому долго училась.

Когда человек рождается, его языковые познания близки к нулевым. Вначале он проговаривает простые звуки, но с возрастом он учится говорить короткие слова, а его словарный запас растёт вместе со способностью складывать эти слова в предложения.

У языковой модели тоже есть словарный запас, который называется *словарём*. Современные языковые модели анализируют большое количество текстов из самых разных источников, тем самым пополняя свой словарь. Такой анализ текстов называется *обучением мо-*

В лесу родилась ёлочка

Слово	Вероятность
абажур	0 %
абакус	0 %
...	...
ёлочка	97 %
...	...
ящерица	1 %
ящур	0%

Рис. 4: Пример выбора следующего слова.

дели. В ходе обучения языковые модели находят закономерности в языке и в структуре предложений, что позволяет им генерировать текст самостоятельно.

Рассмотрим пример. Мы хотим, чтобы компьютер выдал продолжение фразы «В лесу родилась». Если в словаре 80 000 слов, то продолжением фразы может быть любое из 80 000 слов. Как же выбрать наиболее подходящее?

Идея довольно простая: для каждого слова в словаре мы посчитаем вероятность встретить его после «В лесу родилась», а из получившегося распределения выберем наиболее вероятное продолжение фразы. Модели, которые делают вывод о следующем слове по вероятностному распределению, называются *вероятностными моделями*.

Для подсчёта вероятности модель учитывает многое: контекст, встречаемость слова в языке, идиоматику. Может быть, фраза общеизвестна, а может, это фразеологизм. Научить модель точно распределять вероятности между словами – это самая важная и сложная часть создания языковой модели. На этом сайте можно поиграться с настоящим распределением вероятностей для произвольного текста

Enter text:

Never gonna give you up, never gonna let you



12295 8066 1577 345 510 11 1239 8066 1309 345

Prediction

#	probs	next token ID	predicted next token
0	78.85%	866	down
1	12.69%	467	go
2	0.79%	651	get
3	0.68%	510	up
4	0.57%	503	out
5	0.54%	2666	leave
6	0.37%	2121	fall
7	0.30%	307	be
8	0.25%	4656	die
9	0.23%	1011	take

Рис. 5: Скриншот англоязычного ресурса.

на основе англоязычной языковой модели GPT-2.

3.1 Токенизация

В настоящих языковых моделях словари состоят не из слов, а из *токенов* – смысловых единиц текста. Некоторые модели не хранят слова «хороший», «хорошая», «хорошие», а хранят «хорош» и отдельно «ий», «ая», «ие». Это позволяет сделать словарь небольшим, так как он может хранить корни слов и приставки отдельно (см. рис. 6).

Это позволяет языковой модели быть более гибкой в понимании и образовании новых слов. Если раньше слова и предложения хранились и воспринимались программой как набор символов, то теперь они воспринимаются как набор смысловых единиц — *токенов*. Про-



Рис. 6: Пример разбиения текста на токены.

цесс разбиения текста на токены называется *токенизацией*.

Но как понять, как токены должны выглядеть? Если в случае со словарём мы могли все слова, которые нам встречаются в процессе обучения, просто добавлять словарь, то как же проверить подобное с токенами?

Для этого существует алгоритм *byte pair encoding* — *BPE*. Он анализирует, какие сочетания символов встречаются в текстах чаще всего. Предполагается, что если сочетание символов встречается часто, то это сочетание несёт в себе смысл, а значит, можно хранить его отдельным токеном.

Например, приставка «недо» встречается в текстах очень часто и при добавлении этой приставки к слову, его смысл может сильно измениться:

делать → *недо*делать.

3.2 Вероятности и цепи Маркова

Мы уже поняли, что задача генерации текста сводится к задаче распределения вероятностей. Модель генерирует текст токен за токеном. Задача модели — по уже сгенерированному набору токенов сделать вывод о том, какой токен будет следующим. Первый токен обозначим за x_0 . Например, модели нужно продолжить фразу

«Спички детям не ...»,

тогда $x_0 = \text{«Спички»}$.

Запишем эту задачу формально. Пусть X_i — случайная величина, обозначающая i -ый токен в тексте, а x_i — реализация случайной величины X_i , то есть токен, который выбран среди всех возможных токенов на i -ю позицию. Если модель уже сгенерировала $k - 1$ токенов, то вероятность того, что k -й токен — это x_k , считается следующим образом:

$$\mathbb{P}(X_k = x_k | X_0 = x_0, X_1 = x_1, \dots, X_{k-1} = x_{k-1}). \quad (1)$$

Для краткости будем записывать это так:

$$\mathbb{P}(x_k | x_0, x_1, \dots, x_{k-1}). \quad (2)$$

Модель отвечает на вопрос «насколько вероятно, что k -й токен — это именно x_k ?». Оттого, насколько точно будет высчитываться эта вероятность, зависит точность модели. Обучение больших языковых моделей занимает огромное количество ресурсов, и практически все эти ресурсы направлены на минимизацию ошибок при подсчёте вероятности.

Предположим, что в нашем словаре всего 80 000 токенов. Тогда при продолжении фразы «Спички детям не ...» модель должна выбрать один из этих 80 000 токенов. Для этого она оценивает вероятность каждого из них с помощью приведённой выше формулы. Помним, что токеном может быть не только слово, но и его часть, а также пробелы и знаки препинания. Например,

$$\mathbb{P}(\text{абажур} | \text{Спички}, _, \text{детям}, _, \text{не}, _) = 0.00. \quad (3)$$

Вероятность того, что продолжением фразы является «абажур», крайне низкая, несмотря на то, что спички в действительности детям не абажур. Модель такой вариант отсекает. А вариант «игрушка» или «игрушки» будет наиболее вероятным:

$$\mathbb{P}(\text{игрушка} | \text{Спички}, _, \text{детям}, _, \text{не}, _) = 0.75. \quad (4)$$

$$\mathbb{P}(\text{игрушки} | \text{Спички}, _, \text{детям}, _, \text{не}, _) = 0.25. \quad (5)$$

Как же научить языковую модель подсчитывать такие вероятности? Над этим вопросом задумывался великий русский математик *Андрей Андреевич Марков* ещё в 1913 году. Он предположил, что вероятность следующего токена можно оценить, опираясь не на все токены позади, а только на несколько последних:

$$\mathbb{P}(X_k = x_k | X_t = x_t, X_{t+1} = x_{t+1}, \dots, X_{k-1} = x_{k-1}). \quad (6)$$

Такое свойство называется *марковским*. Марков применял это свойство, чтобы предсказать, будет ли следующая буква в отрывке из «Евгения Онегина» гласной или согласной. Проанализировав последовательность из 20 000 букв из «Евгения Онегина», Марков заключил, что вероятность того, что

- после гласной идёт гласная, равна 0.13,
- после гласной идёт согласная, равна 0.87,
- после согласной идёт согласная, равна 0.33,
- после согласной идёт гласная, равна 0.67.

Он предложил схематично это обозначить в виде ориентированного взвешенного графа (см. рис. 7).



Рис. 7: Цепь Маркова.

Такая математическая модель называется *цепью Маркова*. Узлы графа «Гласная» и «Согласная» называются *состояниями*, а рёбра — переходами. Вес ребра показывает вероятность перехода из одного состояния в другое. Весьма удобная и понятная система.

Но мы же предсказываем не следующую букву, а следующее слово, а точнее, токен. Строить такие графы с состояниями в виде токенов и переходами — очень сложно и нецелесообразно. Вместо этого мы напишем программу, которая численно всё подсчитает самостоятельно.

Программа считает все вероятности приближённо. Самый удобный способ для подсчёта условной вероятности

$$\mathbb{P}(x_k | x_t, x_{t+1}, \dots, x_{k-1}), \quad (7)$$

это приблизить её с помощью определения вероятности — отношения количества благоприятствующих событию исходов к количеству всех исходов:

$$\mathbb{P}(x_k | x_t, x_{t+1}, \dots, x_{k-1}) \approx \frac{N(x_t, x_{t+1}, \dots, x_{k-1}, x_k)}{N(x_t, x_{t+1}, \dots, x_{k-1})}, \quad (8)$$

где $N(S)$ — количество вхождений подпоследовательности S в исходных текстах.

Получился достаточно простой алгоритм генерации текста. Мы воспользовались только марковским предположением о том, что достаточно смотреть не слишком далеко вглубь текста для предсказания следующего токена, а также приближённым подсчётом вероятности.

Далее алгоритм можно совершенствовать. Например, на каждом этапе выбирать токен не с самой высокой вероятностью, а случайный из первых 10 наиболее вероятных токенов. Это сделает текст более живым и менее предсказуемым.

Эффективность и качество описанного алгоритма можно оценить в этом ноутбуке.

3.3 Рекуррентные нейронные сети

Цепи Маркова, несмотря на всю логичность идеи, редко позволяют добиться осмысленного текста, а уж о том, чтобы они выполняли какие-то задачи, не приходится и мечтать.

Становится понятно, что для того чтобы сделать текст осмысленным, привести в него логические рассуждения, нужно применять технологии *машинного обучения*. В основном курсе уже обсуждались широкие возможности нейронных сетей. Можно ли их применить для обработки текста?

Давайте изобразим простую нейронную сеть (см. рис. 8).

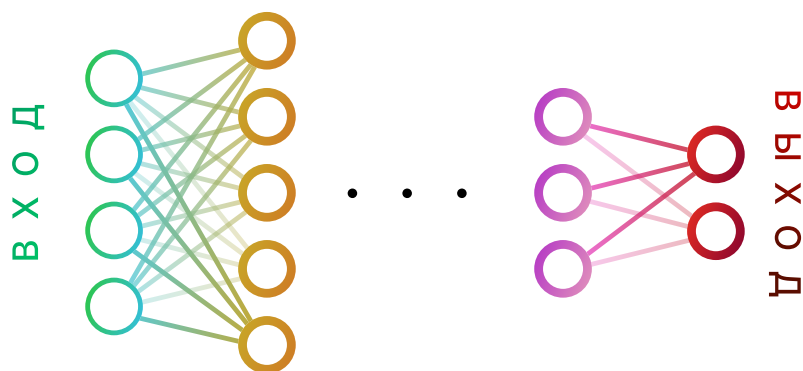


Рис. 8: Простая нейронная сеть (многослойный персептрон).

Размерность входа — 4, а выхода — 2. Мы не можем передать на вход ни больше, ни меньше, а также не можем получить на выходе ни больше, ни меньше. А это сильно ограничивает область применимости обычных нейронных сетей. Мы не можем обрабатывать

последовательные цепочки как, например, аудио, поскольку звуковые дорожки могут иметь разную длительность. Если мы строим модель для распознавания голоса, то такая архитектура нам не подходит. Не подходит она и для обработки текста по той же причине: тексты могут быть самой разной длины.

В 1982 году Джон Хопфилд придумал, как решить эту проблему, и изобрёл *рекуррентные нейронные сети* (*recurrent neural network* — *RNN*). Их особенность заключается в том, что они могут последовательно принимать на вход кусочки входных данных. Взгляните на схему на рис. 9.

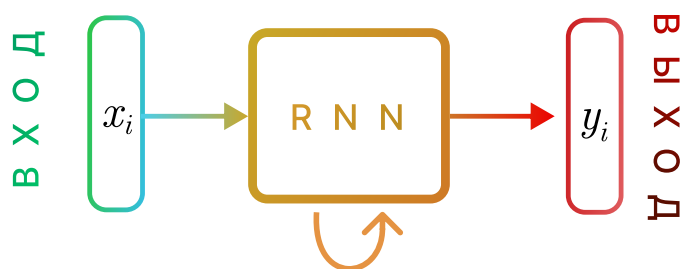


Рис. 9: Иллюстрация рекуррентной архитектуры.

Мы видим, как на вход подаётся i -й кусочек данных — например, очередное слово (токен) текста. Он попадает в рекуррентный слой, который подписан как *RNN*. В результате обработки формируется *скрытое состояние* — это внутренний результат работы слоя. Оно называется скрытым, потому что напрямую не выдаётся пользователю. Чаще всего скрытое состояние содержит в себе всю необходимую информацию для модели, но в сыром виде. Чтобы получить понятный для пользователя ответ, это состояние проходит ещё через несколько преобразований, в результате которых формируется итоговый вывод.

Обработка i -го токена завершена. Однако обратите внимание на стрелку под рекуррентным слоем. В ней заключена главная идея: посчитанное скрытое состояние не теряется, а передаётся обратно в слой для работы со следующим токеном. Теперь при обработке $(i+1)$ -го кусочка данных нейросеть будет учитывать *память* о предыдущем шаге. Это позволяет обрабатывать данные не как набор случайных символов, а как единую связную последовательность.

В каком формате хранятся токены? В машинном обучении данные удобно представлять в виде векторов, которые хранят полезную для модели информацию о смысле слова (или токена). Такое вектор-

ное представление слова (или токена) называется *эмбедингом*:

$$\text{«мир»} := (0.72, -1.28, 2.07, 1.34, -2.91, \dots, -1.22, 4.41).$$

Интересно, что эмбединги слов, схожих по смыслу, похожи, то есть расстояние между ними небольшое. Очень часто, говоря про эмбединги, упоминают интересный пример: если взять эмбединг для слова «король», вычесть из него эмбединг для слова «мужчина», а затем прибавить эмбединг слова «женщина», то получится эмбединг, очень близкий к слову «королева»:

$$\text{«король»} - \text{«мужчина»} + \text{«женщина»} \approx \text{«королева»}. \quad (9)$$

Таким образом, эмбединги способны хранить смысловые составляющие слов, и чем больше размерность эмбединга, тем точнее эмбединг описывает конкретное слово. Технология преобразования слов в эмбединги называется *word2vec* и была изобретена в 2013 году исследователями из *Google*.

В современных языковых моделях эмбединги — это очень большие векторы из тысяч чисел. Размерность вектора эмбединга определяется архитектурой модели, и если она равна 7 000, то все токены кодируются 7 000-мерными векторами, а затем уже обрабатываются нейронными сетями, например, рекуррентными.

Рекуррентные нейронные сети хорошо себя зарекомендовали для решения таких задач как распознавание текста, описание изображения, предсказание следующего токена. Однако с последним возникла большая проблема: рекуррентные сети плохо справляются с очень длинными текстами. Эта проблема несёт название «*проблема затухающего градиента*».

Известно, что для обучения модели необходимо корректировать её параметры, чтобы минимизировать ошибки. Величина, на которую нужно изменить параметр, зависит от *градиента* (градиент также называют *производной*). Чем больше градиент, тем сильнее нужно изменить параметр при обучении. При обучении градиенты последних шагов перемножаются между собой, чтобы получить сигнал для корректировки первых шагов. Если эти значения меньше единицы, то при многократном умножении они устремятся к нулю. Чем длиннее текст, тем меньше становится градиент в начале цепочки — отсюда и название «*затухающий градиент*». Как результат, модель к концу текста забывает его начало.

И хотя эта проблема была решена в 1997 году более усовершенствованной версией рекуррентных нейронных сетей — *длинной це-*

пью элементов краткосрочной памяти (*long short-term memory* — *LSTM*), оставалась ещё одна фундаментальная трудность: нельзя *распараллелить* обучение. То есть, проводить параллельные вычисления разными компьютерами. Такое ограничение возникает из-за последовательной структуры рекуррентной нейронной сети — там просто нечему считаться параллельно. Мы не можем начать вычисления для второго токена, пока не закончим с первым. Это делает обучение на огромных массивах данных очень долгим.

В 2014 году исследователи из *Google Brain* предложили усовершенствованный подход и представили миру *механизм внимания*.

3.4 Механизм внимания

Проблемы, связанные с рекуррентными нейронными сетями и *LSTM*, ограничивали развитие языковых моделей, и принципиально новый подход — *механизм внимания* — изменил историю их развития.

Механизм внимания позволяет модели строить взаимосвязи между словами в предложении, чтобы сформировать контекст. Возьмём в качестве примера предложение:

«Хозяин говорил обтекаемо и настолько деликатно, что его присутствие почти не ощущалось.»

Обратите внимание на слово «его». В то время как обычные рекуррентные архитектуры могли бы *забыть* начало предложения за обилием описательных слов, механизм внимания позволяет модели обнаружить связь слова «его» со словом «*хозяин*», благодаря чему модель лучше *усваивает* контекст. Лишние слова для механизма не проблема, и это его сильная сторона, за счёт которой модель учится определять, на что в тексте нужно обратить особое внимание (отсюда и название).

Как же модель умудряется уловить взаимосвязь между словами? Для того чтобы модель могла посчитать связь между двумя токенами, она предварительно для всех токенов считает три вектора:

- **Query** — запрос

Вектор запроса для токена содержит информацию о том, что данный токен *хочет найти* у других токенов в предложении.

- **Key** — ключ

Вектор ключа предоставляет ответ на запрос. Он помогает определить, насколько этот токен релевантен запросу.

- *Value* — значение

Вектор значения содержит в себе необходимую смысловую информацию о токене.

Чтобы было легче понять, разберём алгоритм внимания на примере. Рассмотрим предложение:

«Мир хранит наш покой, пока мы бережём его.»

Для того чтобы модель нашла взаимосвязи между всеми словами, нужно применить алгоритм внимания для каждой пары слов.

В нашем предложении есть пробелы и знаки препинания, а они тоже являются токенами, как и слова. Однако для упрощения, мы их в рассуждениях опустим.

Рассмотрим пример со словом «его». К чему относится это местоимение? К слову «мир» или «покой»? Человеку сразу понятно, что речь про «мир». Что же делает модель?

Во-первых, модель посчитает векторы запроса, ключа и значения для каждого токена в предложении. Во-вторых, она начнёт работать с парами слов «его» и «мир», а также с «его» и «покой».

Обозначим за $Q_{\text{его}}$ запрос слова «его», $K_{\text{мир}}$ — ключ слова «мир» и $K_{\text{покой}}$ — ключ слова «покой»:

$$\begin{aligned} Q_{\text{его}} &= (-1.11, 0.40, 0.18, \dots, -0.96, -0.14, 0.74, -1.19, 1.73) \\ K_{\text{мир}} &= (0.59, -0.90, 2.38, \dots, 0.15, -1.01, 1.26, 1.19, 0.41) \\ K_{\text{покой}} &= (0.46, -1.35, 0.34, \dots, -0.67, -1.32, 1.41, -0.69, 0.09). \end{aligned} \quad (10)$$

Модели нужно определить, к какому слову относится слово «его», поэтому слово «его» задаёт вопрос $Q_{\text{его}}$ — «у кого есть нужная мне информация?». Слово «мир» отвечает: его ответ — это ключ $K_{\text{мир}}$. Также отвечает слово «покой»: $K_{\text{покой}}$.

Осталось проверить, чей же ответ нам более релевантен. Делается это с помощью скалярного произведения. Посчитаем

$$Q_{\text{его}} \cdot K_{\text{мир}} = 142.21 \quad (11)$$

и

$$Q_{\text{его}} \cdot K_{\text{покой}} = -34.33. \quad (12)$$

Скажем, что слово «мир» более релевантно для слова «его», чем слово «покой», поскольку соответствующее скалярное произведение оказалось больше.

Получившиеся числа в целях нормализации делятся на $\sqrt{d_k}$, где d_k — размерность вектора ключа, ведь если размерность вектора будет большой, результат скалярного произведения тоже может быть большим, что может негативно сказаться на процессе обучения. Мы для примера будем считать, что $d_k = 7\,000$:

$$\begin{aligned} Q_{\text{его}} \cdot K_{\text{мир}} &= 142.21 & \Rightarrow & Q_{\text{его}} \cdot K_{\text{мир}} / \sqrt{d_k} = 1.69 \\ Q_{\text{его}} \cdot K_{\text{покой}} &= -34.33 & \Rightarrow & Q_{\text{его}} \cdot K_{\text{покой}} / \sqrt{d_k} = -0.41. \end{aligned} \quad (13)$$

Подобным образом найдём связи слова «его» со всеми остальными словами в тексте:

$$\begin{aligned} Q_{\text{его}} \cdot K_{\text{хранит}} &= -50.14 & \Rightarrow & Q_{\text{его}} \cdot K_{\text{хранит}} / \sqrt{d_k} = -0.59 \\ Q_{\text{его}} \cdot K_{\text{наш}} &= -80.51 & \Rightarrow & Q_{\text{его}} \cdot K_{\text{наш}} / \sqrt{d_k} = -0.96 \\ Q_{\text{его}} \cdot K_{\text{пока}} &= -120.23 & \Rightarrow & Q_{\text{его}} \cdot K_{\text{пока}} / \sqrt{d_k} = -1.43 \\ Q_{\text{его}} \cdot K_{\text{мы}} &= -15.40 & \Rightarrow & Q_{\text{его}} \cdot K_{\text{мы}} / \sqrt{d_k} = -0.18 \\ Q_{\text{его}} \cdot K_{\text{бережём}} &= 10.55 & \Rightarrow & Q_{\text{его}} \cdot K_{\text{бережём}} / \sqrt{d_k} = 0.12 \\ Q_{\text{его}} \cdot K_{\text{его}} &= 45.03 & \Rightarrow & Q_{\text{его}} \cdot K_{\text{его}} / \sqrt{d_k} = 0.53. \end{aligned} \quad (14)$$

Как модель смогла наделить векторы запроса и ключей такими значениями, чтобы скалярное произведение отражало релевантность? Ответ кроется в методе получения векторов $Q_{\text{его}}$, $K_{\text{мир}}$ и $K_{\text{покой}}$.

Модель имеет три матрицы: W_Q , W_K и W_V , отвечающие за запрос, ключ и значение соответственно. Чтобы получить вектор запроса для токена «его», нужно умножить эмбединг токена «его», который обозначим за $v_{\text{его}}$, на матрицу W_Q :

$$Q_{\text{его}} = W_Q v_{\text{его}}. \quad (15)$$

Аналогично посчитаем вектор ключа, например, для слова «мир»:

$$K_{\text{мир}} = W_K v_{\text{мир}}. \quad (16)$$

Матрицы W_Q , W_K и W_V являются *обучаемыми*. Это означает, что в процессе обучения языковой модели, в процессе чтения и обработки огромных корпусов текстов эти матрицы настроились так, чтобы вектор запроса максимально точно формулировал *вопрос*, а вектор ключа содержал *правильный ответ*.

Для чего же нужна матрица W_V ? Она преобразовывает эмбединг токена так, чтобы он давал всю необходимую информацию о токене:

$$V_{\text{мир}} = W_V v_{\text{мир}}. \quad (17)$$

В результате мы получим *вектор значения*. Теоретически, эмбединг тоже содержит в себе смысловую информацию о токене, но она слишком общая, в то время как вектор значения содержит в себе только ту *отфильтрованную* матрицей W_V информацию, которая релевантна для текущей задачи.

Теперь, когда мы знаем векторы значений всех слов, нам осталось собрать *смысл* слова «его». Мы уже оценили вклад каждого слова в *смысл* слова «его» скалярным произведением, делённым на корень из размерности вектора ключа, однако удобнее будет представить полученные числа в виде вероятностного распределения. То есть нам интереснее знать, на сколько процентов данное слово важно для слова «его». В этом нам поможет оператор *softmax*:

$$\text{softmax}(\mathbf{x})_j = \frac{e^{x_j}}{\sum_{i=1}^n e^{x_i}}. \quad (18)$$

Не будем заострять на нём внимание. Ранее мы оценивали значимость слова «мир» в 1.69, а после преобразования получим 0.496 (49.6%). Посчитаем и для каждого слова (см. рис. 10).

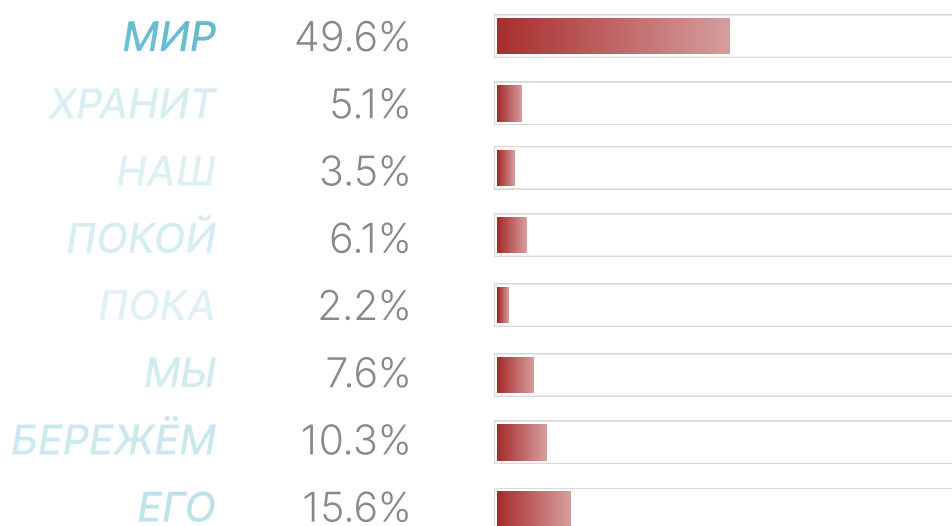


Рис. 10: Визуализация весов внимания.

В этой таблице заключается вся идея механизма внимания. Модель посчитала для слова «его» релевантность каждого слова. Сразу

видно, что слово «**мир**» наиболее релевантно, а значит, под «**его**» скорее всего подразумевается «**мир**».

Мы хотим, чтобы наиболее релевантные слова вносили наибольший вклад в *смысл* слова, а наименее релевантные — наименьший. Для этого посчитаем взвешенную сумму:

$$A_{\text{его}} = \sum_{i=1}^8 \alpha_i V_i, \quad (19)$$

где $A_{\text{его}}$ — искомый *вектор внимания* для слова «**его**», V_i — вектор значения i -го слова, а α_i — оценённая релевантность (это число от 0 до 1; например, для слова «**мир**» релевантность будет равна 0.496). Значения α_i называют *весами внимания*.

Таким образом, сумма имеет вид

$$A_{\text{его}} = 0.496V_{\text{мир}} + 0.051V_{\text{хранит}} + 0.035V_{\text{наш}} + \dots + 0.156V_{\text{его}}. \quad (20)$$

Полученный вектор является результатом применения алгоритма внимания для слова «**его**». Теперь он будет использоваться как эмбединг слова «**его**», поскольку он очень точно описывает его смысл.

Таким образом, мы описали идею, что токены с большей релевантностью будут сильнее влиять на контекстное представление (вектор внимания), чем токены с меньшей релевантностью. Вот и вся идея механизма внимания — более совершенная и элегантная.

Механизм внимания, применённый для определения связи между словами одного и того же предложения, называется механизмом *самовнимания* (*self-attention*).

Однако для генерации текста на одном механизме внимания далеко не уедешь. Или уедешь?..

3.5 Внимание — всё, что вам нужно

В 2017 году исследователи из *Google* опубликовали статью “*Attention is All You Need*” («Внимание — всё, что вам нужно»), которая произвела революцию в области обработки текста.

До выхода этой статьи на свет в мире искусственного интеллекта доминировали две технологии работы с текстом, о которых мы уже говорили: *RNN* и *LSTM*. Эти модели обрабатывали текст последовательно, слово за словом, запоминая информацию о предыдущих словах во внутреннем состоянии.

После выхода в 2014 году статьи о механизме внимания появилась возможность улучшить технологии *RNN* и *LSTM* внедрением в них механизма внимания. Это привело к заметному росту качества, особенно в задачах перевода.

Несмотря на это, оставалась нерешённой серьёзная проблема: такие модели по-прежнему работали последовательно. Это означало, что их обучение было медленным, а эффективно распараллелить вычисления практически невозможно. Из-за этого обучение требовало слишком больших вычислительных мощностей и большого запаса терпения.

Однако в статье 2017 года “*Attention is All You Need*” появилась принципиально новая технология работы с текстом и изображениями, которая получила название **трансформер**. Особенность новой архитектуры заключается в том, что она не содержит никаких рекуррентных слоёв: она основана исключительно на механизме внимания, в отличие от, например, рекуррентных нейронных сетей, где механизм внимания — лишь дополнение к основной архитектуре. Отсутствие рекуррентности позволило распараллелить обучение, за счёт чего обучение стало значительно более быстрым.

Классическая архитектура трансформера состоит из двух частей: **энкодера** и **декодера** (см. рис. 11).

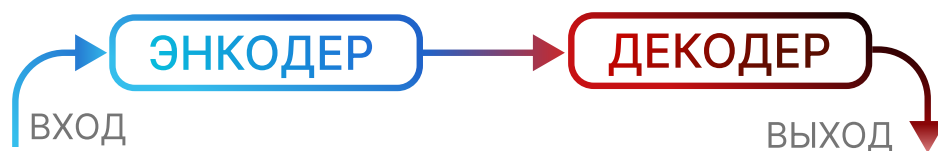


Рис. 11: Классическая архитектура трансформера, состоящая из **энкодера** и **декодера**.

Энкодер — это часть архитектуры, которая анализирует входной текст и превращает его в цифровое представление — набор векторов, в которых заложен смысл каждого слова. Затем энкодер передаёт полученные представления **декодеру**, который, в свою очередь, поэтапно строит выходной текст.

Мы не будем подробно рассматривать устройство энкодера и декодера, но затронем основные идеи.

В энкодере механизм самовнимания применяется несколько раз параллельно и независимо. Говорят, что *входные данные обрабатываются несколькими головами внимания одновременно*. Каждая *голова внимания* реализует механизм внимания и работает со своим

набором параметров. У каждой головы есть собственные матрицы W_Q , W_K и W_V , благодаря чему разные головы могут по-разному анализировать один и тот же текст и выделять различные зависимости между словами.

Декодер, в свою очередь, выполняет генерацию текста слово за словом (хотя точнее сказать *токен за токеном*). На каждом шаге декодер получает на вход:

- уже сгенерированные декодером токены;
- представления входного текста, полученные от энкодера.

Модель применяет механизм самовнимания для определения связи между токенами, которые она уже сгенерировала. После обработки уже сгенерированных токенов модель уже обращается к информации, полученной от энкодера с помощью механизма внимания.

После всех применённых механизмов внимания у модели появляется векторное представление каждого токена. Векторное представление последнего токена передаётся в самую обычную нейронную сеть — в полносвязный слой. На выходе нейронная сеть предоставляет распределение вероятностей для каждого токена, подобно тому, как это работает в алгоритме генерации с цепями Маркова.

Вот мы концептуально и разобрали каждый шаг. Это описание в оригинальной статье было изображено так: (см. рис. 12).

Изначально трансформер разрабатывался для решения задачи машинного перевода. Сейчас же архитектура трансформера получила широкое распространение и в генерации текста чат-ботами, и в генерации изображений и видео. Например, чат-бот *ChatGPT* основан на одной из вариаций архитектуры трансформера

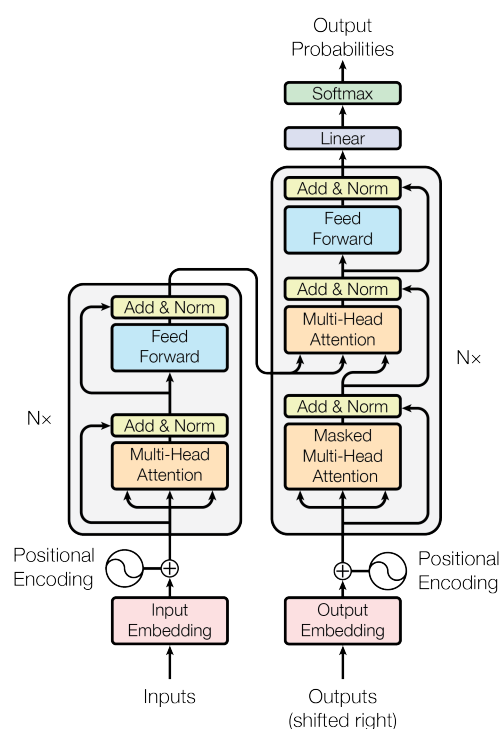


Рис. 12: Иллюстрация из оригинальной статьи.

– **GPT** (*Generative Pretrained Transformer*), что означает «генеративный предобученный трансформер».

Называется он так потому, что он выполняет задачу генерации текста (отсюда слово «генеративный»), он обучен на большом количестве корпусов текста (отсюда слово «предобученный»), а также что в его основе лежит архитектура трансформера (отсюда слово «трансформер»).

Вариация архитектуры *ChatGPT* отличается от классической отсутствием энкодера. Архитектура без энкодера называется *decoder-only* (только декодер), то есть модель занимается только генерацией текста, а не его преобразованием (как, например, в случае с переводчиком, энкодер которого принимает предложение на исходном языке, а декодер переводит его на другой).

Несмотря на то, что модели новой архитектуры легче и быстрее в обучении, процесс разработки языковых моделей всё равно остаётся очень дорогим и ресурсоёмким. Мы рассмотрели только техническую сторону вопроса, хотя в действительности он включает в себя большую работу по обработке и фильтрации обучающих данных, по настройке параметров обучения, по настройке моделей на генерацию правильных ответов, которые с одной стороны будут полезны пользователю, а с другой – будут этически корректными.

4 Заключение

Теперь мы готовы ответить на главный вопрос «как компьютер генерирует текст?». Для компьютера любой текст — набор токенов. Компьютер генерирует текст токен за токеном. Для выбора следующего токена необходимо выбрать наиболее вероятное продолжение к уже существующему сгенерированному набору токенов.

Научить компьютер точному распределению вероятностей – очень трудоёмкий процесс. Если применять марковское свойство для распределения вероятностей, то оно не позволит построить осмысленный текст. Вместо этого применяются более совершенные методы, например, архитектура трансформера.

Языковые модели обучаются на больших текстах. Они могут сами определять правила построения предложений, выявлять закономерности и сложные шаблоны. Разработчики настраивают модели так, чтобы они меньше глючили, давали правильные ответы, и не вели себя некорректно. Вот так компьютер и генерирует текст.